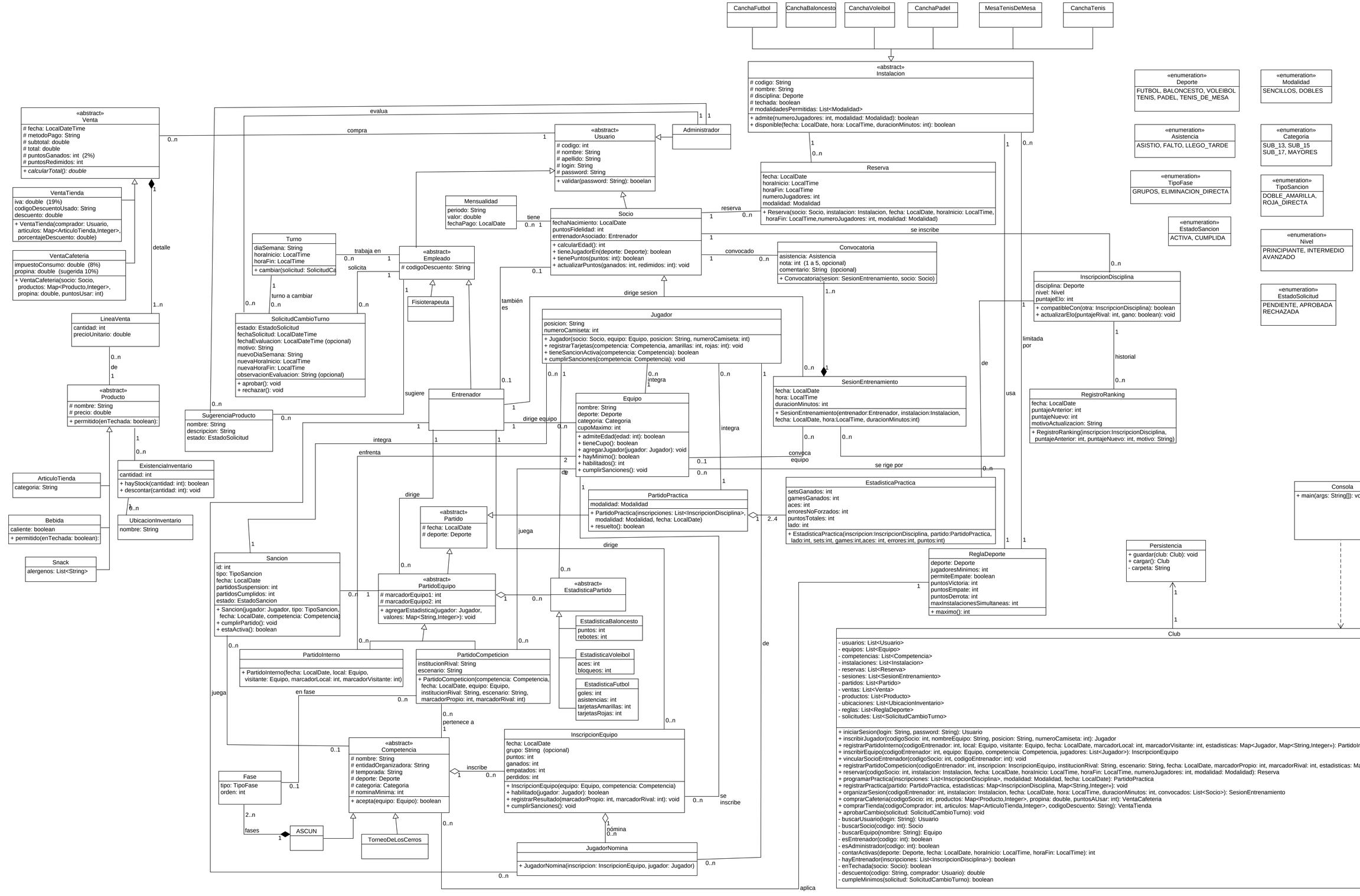
CanchaFutbol
CanchaBaloncesto
CanchaVoleibol
CanchaPadel
MesaTenisDeMesa
CanchaTenis
«abstract»
Instalacion
# codigo: String
# nombre: String
# disciplina: Deporte
# techada: boolean
# modalidadesPermitidas: List<Modalidad>
+ admite(numeroJugadores: int, modalidad: Modalidad): boolean
+ disponible(fecha: LocalDate, hora: LocalTime, duracionMinutos: int): boolean
«enumeration»
Deporte
FUTBOL, BALONCESTO, VOLEIBOL
TENIS, PADEL, TENIS_DE_MESA
«enumeration»
Modalidad
SENCILLOS, DOBLES
«enumeration»
Asistencia
ASISTIO, FALTO, LLEGO_TARDE
«enumeration»
Categoria
SUB_13, SUB_15
SUB_17, MAYORES
«enumeration»
TipoFase
GRUPOS, ELIMINACION_DIRECTA
«enumeration»
TipoSancion
DOBLE_AMARILLA,
ROJA_DIRECTA
«enumeration»
EstadoSancion
ACTIVA, CUMPLIDA
«enumeration»
Nivel
PRINCIPIANTE, INTERMEDIO
AVANZADO
«enumeration»
EstadoSolicitud
PENDIENTE, APROBADA
RECHAZADA
«abstract»
Venta
# fecha: LocalDateTime
# metodoPago: String
# subtotal: double
# total: double
# puntosGanados: int (2%)
# puntosRedimidos: int
+ calcularTotal(): double
compra
evalua
«abstract»
Usuario
# codigo: int
# nombre: String
# apellido: String
# login: String
# password: String
+ validar(password: String): booelan
Administrador
VentaTienda
iva: double (19%)
codigoDescuentoUsado: String
descuento: double
+ VentaTienda(comprador: Usuario,
articulos: Map<ArticuloTienda,Integer>,
porcentajeDescuento: double)
VentaCafeteria
impuestoConsumo: double (8%)
propina: double (sugerida 10%)
+ VentaCafeteria(socio: Socio,
productos: Map<Producto,Integer>,
propina: double, puntosUsar: int)
detalle
LineaVenta
cantidad: int
precioUnitario: double
de
«abstract»
Producto
# nombre: String
# precio: double
+ permitido(enTechada: boolean):
ExistenciaInventario
cantidad: int
+ hayStock(cantidad: int): boolean
+ descontar(cantidad: int): void
ArticuloTienda
categoria: String
Bebida
caliente: boolean
+ permitido(enTechada: boolean):
Snack
alergenos: List<String>
UbicacionInventario
nombre: String
Reserva
fecha: LocalDate
horaInicio: LocalTime
horaFin: LocalTime
numeroJugadores: int
modalidad: Modalidad
+ Reserva(socio: Socio, instalacion: Instalacion, fecha: LocalDate, horaInicio: LocalTime,
horaFin: LocalTime,numeroJugadores: int, modalidad: Modalidad)
reserva
se inscribe
Mensualidad
periodo: String
valor: double
fechaPago: LocalDate
tiene
Socio
fechaNacimiento: LocalDate
puntosFidelidad: int
entrenadorAsociado: Entrenador
+ calcularEdad(): int
+ tieneJugadorEn(deporte: Deporte): boolean
+ tienePuntos(puntos: int): boolean
+ actualizarPuntos(ganados: int, redimidos: int): void
convocado
Convocatoria
asistencia: Asistencia
nota: int (1 a 5, opcional)
comentario: String (opcional)
+ Convocatoria(sesion: SesionEntrenamiento, socio: Socio)
InscripcionDisciplina
disciplina: Deporte
nivel: Nivel
puntajeElo: int
+ compatibleCon(otra: InscripcionDisciplina): boolean
+ actualizarElo(puntajeRival: int, gano: boolean): void
limitada
por
historial
RegistroRanking
fecha: LocalDate
puntajeAnterior: int
puntajeNuevo: int
motivoActualizacion: String
+ RegistroRanking(inscripcion:InscripcionDisciplina,
puntajeAnterior: int, puntajeNuevo: int, motivo: String)
Turno
diaSemana: String
horaInicio: LocalTime
horaFin: LocalTime
+ cambiar(solicitud: SolicitudCa
trabaja en
solicita
turno a cambiar
«abstract»
Empleado
# codigoDescuento: String
Fisioterapeuta
también
es
dirige sesion
Jugador
posicion: String
numeroCamiseta: int
+ Jugador(socio: Socio, equipo: Equipo, posicion: String, numeroCamiseta: int)
+ registrarTarjetas(competencia: Competencia, amarillas: int, rojas: int): void
+ tieneSancionActiva(competencia: Competencia): boolean
+ cumplirSanciones(competencia: Competencia): void
SolicitudCambioTurno
estado: EstadoSolicitud
fechaSolicitud: LocalDateTime
fechaEvaluacion: LocalDateTime (opcional)
motivo: String
nuevoDiaSemana: String
nuevaHoraInicio: LocalTime
nuevaHoraFin: LocalTime
observacionEvaluacion: String (opcional)
+ aprobar(): void
+ rechazar(): void
SesionEntrenamiento
fecha: LocalDate
hora: LocalTime
duracionMinutos: int
+ SesionEntrenamiento(entrenador:Entrenador, instalacion:Instalacion,
fecha: LocalDate, hora:LocalTime, duracionMinutos:int)
usa
sugiere
Entrenador
SugerenciaProducto
nombre: String
descripcion: String
estado: EstadoSolicitud
dirige equipo
integra
Equipo
nombre: String
deporte: Deporte
categoria: Categoria
cupoMaximo: int
+ admiteEdad(edad: int): boolean
+ tieneCupo(): boolean
+ agregarJugador(jugador: Jugador): void
+ hayMinimo(): boolean
+ habilitados(): int
+ cumplirSanciones(): void
convoca
equipo
enfrenta
se rige por
EstadisticaPractica
setsGanados: int
gamesGanados: int
aces: int
erroresNoForzados: int
puntosTotales: int
lado: int
+ EstadisticaPractica(inscripcion:InscripcionDisciplina, partido:PartidoPractica,
lado:int, sets:int, games:int,aces: int, errores:int, puntos:int)
dirige
PartidoPractica
modalidad: Modalidad
+ PartidoPractica(inscripciones: List<InscripcionDisciplina>,
modalidad: Modalidad, fecha: LocalDate)
+ resuelto(): boolean
«abstract»
Partido
# fecha: LocalDate
# deporte: Deporte
juega
ReglaDeporte
deporte: Deporte
jugadoresMinimos: int
permiteEmpate: boolean
puntosVictoria: int
puntosEmpate: int
puntosDerrota: int
maxInstalacionesSimultaneas: int
+ maximo(): int
Persistencia
+ guardar(club: Club): void
+ cargar(): Club
- carpeta: String
Consola
+ main(args: String[]): void
Sancion
id: int
tipo: TipoSancion
fecha: LocalDate
partidosSuspension: int
partidosCumplidos: int
estado: EstadoSancion
+ Sancion(jugador: Jugador, tipo: TipoSancion,
fecha: LocalDate, competencia: Competencia)
+ cumplirPartido(): void
+ estaActiva(): boolean
«abstract»
PartidoEquipo
# marcadorEquipo1: int
# marcadorEquipo2: int
+ agregarEstadistica(jugador: Jugador,
valores: Map<String,Integer>): void
«abstract»
EstadisticaPartido
EstadisticaBaloncesto
puntos: int
rebotes: int
EstadisticaVoleibol
aces: int
bloqueos: int
EstadisticaFutbol
goles: int
asistencias: int
tarjetasAmarillas: int
tarjetasRojas: int
PartidoInterno
+ PartidoInterno(fecha: LocalDate, local: Equipo,
visitante: Equipo, marcadorLocal: int, marcadorVisitante: int)
PartidoCompeticion
institucionRival: String
escenario: String
+ PartidoCompeticion(competencia: Competencia,
fecha: LocalDate, equipo: Equipo,
institucionRival: String, escenario: String,
marcadorPropio: int, marcadorRival: int)
en fase
pertenece a
«abstract»
Competencia
# nombre: String
# entidadOrganizadora: String
# temporada: String
# deporte: Deporte
# categoria: Categoria
# nominaMinima: int
+ acepta(equipo: Equipo): boolean
inscribe
InscripcionEquipo
fecha: LocalDate
grupo: String (opcional)
puntos: int
ganados: int
empatados: int
perdidos: int
+ InscripcionEquipo(equipo: Equipo, competencia: Competencia)
+ habilitado(jugador: Jugador): boolean
+ registrarResultado(marcadorPropio: int, marcadorRival: int): void
+ cumplirSanciones(): void
se
inscribe
Fase
tipo: TipoFase
orden: int
2..n
fases
ASCUN
TorneoDeLosCerros
nómina
JugadorNomina
+ JugadorNomina(inscripcion: InscripcionEquipo, jugador: Jugador)
aplica
Club
- usuarios: List<Usuario>
- equipos: List<Equipo>
- competencias: List<Competencia>
- instalaciones: List<Instalacion>
- reservas: List<Reserva>
- sesiones: List<SesionEntrenamiento>
- partidos: List<Partido>
- ventas: List<Venta>
- productos: List<Producto>
- ubicaciones: List<UbicacionInventario>
- reglas: List<ReglaDeporte>
- solicitudes: List<SolicitudCambioTurno>
+ iniciarSesion(login: String, password: String): Usuario
+ inscribirJugador(codigoSocio: int, nombreEquipo: String, posicion: String, numeroCamiseta: int): Jugador
+ registrarPartidoInterno(codigoEntrenador: int, local: Equipo, visitante: Equipo, fecha: LocalDate, marcadorLocal: int, marcadorVisitante: int, estadisticas: Map<Jugador, Map<String,Integer»): PartidoInterno
+ inscribirEquipo(codigoEntrenador: int, equipo: Equipo, competencia: Competencia, jugadores: List<Jugador>): InscripcionEquipo
+ vincularSocioEntrenador(codigoSocio: int, codigoEntrenador: int): void
+ registrarPartidoCompeticion(codigoEntrenador: int, inscripcion: InscripcionEquipo, institucionRival: String, escenario: String, fecha: LocalDate, marcadorPropio: int, marcadorRival: int, estadisticas: Map<Jugador, Map<String,Integer»): PartidoCompeticion
+ reservar(codigoSocio: int, instalacion: Instalacion, fecha: LocalDate, horaInicio: LocalTime, horaFin: LocalTime, numeroJugadores: int, modalidad: Modalidad): Reserva
+ programarPractica(inscripciones: List<InscripcionDisciplina>, modalidad: Modalidad, fecha: LocalDate): PartidoPractica
+ registrarPractica(partido: PartidoPractica, estadisticas: Map<InscripcionDisciplina, Map<String,Integer»): void
+ organizarSesion(codigoEntrenador: int, instalacion: Instalacion, fecha: LocalDate, hora: LocalTime, duracionMinutos: int, convocados: List<Socio>): SesionEntrenamiento
+ comprarCafeteria(codigoSocio: int, productos: Map<Producto,Integer>, propina: double, puntosAUsar: int): VentaCafeteria
+ comprarTienda(codigoComprador: int, articulos: Map<ArticuloTienda,Integer>, codigoDescuento: String): VentaTienda
+ aprobarCambio(solicitud: SolicitudCambioTurno): void
- buscarUsuario(login: String): Usuario
- buscarSocio(codigo: int): Socio
- buscarEquipo(nombre: String): Equipo
- esEntrenador(codigo: int): boolean
- esAdministrador(codigo: int): boolean
- contarActivas(deporte: Deporte, fecha: LocalDate, horaInicio: LocalTime, horaFin: LocalTime): int
- hayEntrenador(inscripciones: List<InscripcionDisciplina>): boolean
- enTechada(socio: Socio): boolean
- descuento(codigo: String, comprador: Usuario): double
- cumpleMinimos(solicitud: SolicitudCambioTurno): boolean